

# Planning Agent 规划书：基于 Typed Task Graph 的仓库环境预规划模块

## 1. 模块目标

Planning Agent 的目标是在 Build Agent 真正开始构建环境之前，对目标仓库进行一次 **read-only exploration**。它尽量不修改环境、不安装依赖、不启动服务，而是通过读取文件、分析配置、搜索代码和理解文档，提前构建一个仓库环境任务图。

这个任务图不是单纯的 dependency graph，而是一个 **Typed Task Graph**。它把仓库运行所需的内容划分为不同类型的任务节点，例如运行时、包管理器、语言依赖、系统依赖、构建步骤、服务、环境变量、数据资源和验证命令。随后，系统根据这些节点之间的依赖关系生成一个有顺序、有证据、有风险提示的 todo-list，供 Build Agent 执行。

简而言之：

Planning Agent 负责回答：

“这个 repo 要跑起来，大概率需要先准备什么、后执行什么、哪些东西是必须的、哪些东西只是推测的？”

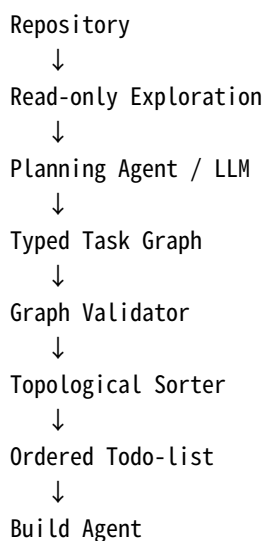
Build Agent 负责回答：

“根据这个计划，我如何在 sandbox 里真正把环境搭起来？”

---

## 2. 整体架构

整体流程如下：



↓  
Sandbox Execution

其中：

Planning Agent 负责 repo 理解和 graph 构建。

Graph Validator 负责检查 graph 是否合法。

Topological Sorter 负责对 graph 做确定性排序。

Build Agent 根据 todo-list 在 sandbox 中真正执行环境构建。

核心原则是：

**LLM 负责理解 repo，程序负责保证排序正确性。**

---

## 3. 输入与输出

### 3.1 输入

Planning Agent 的输入包括：

1. 仓库路径或仓库 URL；
2. 仓库文件结构；
3. README、docs、CI 配置、Dockerfile、配置文件、依赖文件、测试目录；
4. 可选：目标验证方式，例如 pytest collect、unit test、build check、smoke test。

### 3.2 输出

Planning Agent 输出一个 `EnvironmentBuildPlan`。

建议结构如下：

```
{
  "repo_summary": {},
  "typed_task_graph": {
    "nodes": [],
    "edges": []
  },
  "ordered_todo_list": [],
  "risk_notes": [],
  "fallback_plan": [],
  "unresolved_questions": []
}
```

其中：

`repo_summary` 用来总结仓库的语言、包管理器、测试框架、构建工具和可能的 base image。

`typed_task_graph` 是核心结构，表示 repo 环境依赖的任务图。

`ordered_todo_list` 是给 Build Agent 的执行计划。

`risk_notes` 记录可能出错的地方，例如 Python 版本不兼容、lockfile 不能绕过、可能缺少系统库。

`fallback_plan` 记录某些失败发生后的修复建议。

`unresolved_questions` 记录 Planning Agent 无法仅凭只读探索确定的问题，例如是否需要真实 API key、是否需要外部数据集。

---

## 4. Read-only 原则

Planning Agent 只允许执行只读命令，例如：

```
ls
find
cat
sed -n
head
tail
grep
rg
tree
python -c "只做文件解析，不安装包"
```

Planning Agent 不允许执行以下操作：

```
pip install
npm install
apt-get install
poetry install
mvn install
cargo build
pytest
docker run
service start
```

原因是 Planning Agent 的职责不是试错，而是先形成环境地图，避免 Build Agent 盲目探索和污染环境。

如果必须使用某些工具解析文件，也应该优先使用不修改环境的方式，例如解析：

```
pyproject.toml
package.json
```

```
pom.xml
go.mod
Cargo.toml
requirements.txt
Dockerfile
docker-compose.yml
.github/workflows/*.yml
```

而不是直接安装依赖。

## 5. Typed Task Graph 设计

Typed Task Graph 由两部分组成：

```
nodes: 环境任务或资源
edges: 任务之间的依赖关系
```

节点表示“需要准备什么”。

边表示“谁必须在谁之前完成”。

例如：

```
python:3.11-slim
  ↓
pip
  ↓
requirements.txt
  ↓
pytest --collect-only -q
```

这里的每个方框都是一个 node，每个箭头都是一条 edge。

## 6. 节点类型设计

建议先支持以下 9 类节点，不要一开始做得太复杂。

### 6.1 Runtime 节点

Runtime 节点表示 repo 运行所依赖的基础执行环境。

它不是具体的 Python 包，也不是 `pip install` 安装的依赖，而是更底层的语言版本、系统环境或硬件环境。

例子：

```
{
  "id": "python>=3.10",
  "type": "runtime",
  "evidence": ["pyproject.toml"],
  "confidence": 0.9
}
```

常见内容包括：

Python 版本  
Node 版本  
JDK 版本  
Go 版本  
Rust toolchain  
Ubuntu / Debian base image  
CUDA 版本

例如：

```
python:3.10-slim
node:20
openjdk:17
rust:1.75
ubuntu:22.04
pytorch/pytorch:2.1.0-cuda11.8-cudnn8-runtime
```

Runtime 节点解决的是：

**Build Agent 一开始应该选什么 base image、language version 或 toolchain。**

如果 Runtime 选错了，后面再怎么安装依赖都可能失败。例如项目需要 Python 3.10，但 base image 选择了 Python 3.8，就可能出现：

```
Package requires Python >=3.10
No matching distribution found
SyntaxError
```

---

## 6.2 Package Manager 节点

Package Manager 节点表示 repo 应该使用哪个包管理器。

例子：

```
{
  "id": "poetry",
  "type": "package_manager",
  "evidence": ["poetry.lock", "pyproject.toml"],
  "confidence": 0.95
}
```

常见包管理器包括：

```
pip
poetry
conda
npm
yarn
pnpm
maven
gradle
cargo
go mod
```

Package Manager 节点解决的是：

**依赖应该通过什么工具安装。**

例如，看到 `poetry.lock` 时，Build Agent 不应该一开始就绕过 Poetry 直接 `pip install`，否则可能破坏 lockfile 约束。

---

## 6.3 Language Dependency 节点

Language Dependency 节点表示语言层面的依赖包。

例子：

```
{
  "id": "pytest",
  "type": "language_dependency",
  "scope": "test",
  "evidence": ["pyproject.toml", "tests/"],
  "confidence": 0.85
}
```

这里要区分三类依赖：

runtime dependency: 项目运行需要  
dev/test dependency: 测试或开发需要  
optional dependency: 可选功能需要

例如 Python repo 里:

```
flask  
numpy  
requests  
pytest  
pytest-cov
```

Node repo 里:

```
express  
typescript  
jest  
eslint
```

Java repo 里:

```
junit  
spring-boot  
maven-surefire-plugin
```

---

## 6.4 System Package 节点

System Package 节点表示操作系统层面的依赖。

例子:

```
{  
  "id": "libpq-dev",  
  "type": "system_package",  
  "evidence": ["import psycopg2", "PostgreSQL usage in README"],  
  "confidence": 0.7  
}
```

常见系统依赖包括:

```
build-essential  
gcc  
g++
```

```
make
cmake
pkg-config
libssl-dev
libffi-dev
libpq-dev
libxml2-dev
libxslt1-dev
ffmpeg
libgl1
```

这一类非常重要，因为很多 `pip install`、`npm install`、`mvn package` 失败，本质不是语言包缺失，而是缺少系统库或编译工具。

例如：

```
pip install psycopg2
可能需要：
apt-get install libpq-dev
```

```
pip install lxml
可能需要：
apt-get install libxml2-dev libxslt1-dev
```

```
pip install opencv-python
可能需要：
apt-get install libgl1
```

---

## 6.5 Project Build 节点

Project Build 节点表示项目自身的安装、编译或构建步骤。

例子：

```
{
  "id": "poetry install",
  "type": "project_build",
  "evidence": ["poetry.lock", "README.md"],
  "confidence": 0.9
}
```

常见步骤包括：



```
pip install -e .  
poetry install  
npm run build  
mvn package  
gradle build  
cargo build  
make  
cmake build
```

注意：不是所有 repo 都需要 `pip install -e .`。

如果 CI 或 tox 明确设置了 `skip_install = true`，或者测试是直接在源码树上运行，那么 Project Build 节点可以是：

```
source-tree execution
```

意思是：项目不需要被安装成 package，只要依赖安装好，就可以在源码目录直接运行测试。

## 6.6 Service 节点

Service 节点表示测试或实际运行时需要启动的外部服务。

例子：

```
{  
  "id": "redis-server",  
  "type": "service",  
  "evidence": ["tests mention redis", "docker-compose.yml"],  
  "confidence": 0.75  
}
```

常见服务包括：

```
Redis  
Postgres  
MySQL  
MongoDB  
Kafka  
RabbitMQ  
Elasticsearch  
Mosquitto  
Selenium  
Playwright browser
```

Service 节点需要和普通 dependency 区分开。

例如：

```
redis Python package
```

只是 Python 访问 Redis 的客户端库。

而：

```
redis-server
```

才是真正需要启动的 Redis 服务。

---

## 6.7 Env Var / Secret 节点

Env Var / Secret 节点表示环境变量、配置变量、API key 或 token。

例子：

```
{
  "id": "OPENAI_API_KEY",
  "type": "env_var",
  "evidence": ["os.environ['OPENAI_API_KEY']"],
  "confidence": 0.95,
  "requires_real_secret": true
}
```

这里需要判断：

```
是否有 .env.example
是否只是测试中 mock 掉
是否必须真实 secret
是否可以用 dummy value
```

如果某个变量必须是真实 API key，Planning Agent 不应该让 Build Agent 盲目尝试，而应该把它标记为 unresolved external resource。

---

## 6.8 Data / Asset 节点

Data / Asset 节点表示数据集、模型权重、浏览器、fixture 文件等外部资源。

例子：

```
{
  "id": "chromium-browser",
  "type": "asset",
  "evidence": ["playwright usage"],
  "confidence": 0.8
}
```

常见资源包括：

```
测试数据
模型文件
浏览器 binary
NLTK data
spaCy model
HuggingFace model
sample database
fixture file
```

这类资源经常不在 package manifest 里，需要从 README、测试代码、CI 和错误日志中推断。

---

## 6.9 Runtime Command 节点

Runtime Command 节点表示项目实际运行时需要执行的命令。

例子：

```
{
  "id": "python proxyPool.py server",
  "type": "runtime_command",
  "evidence": ["README.md"],
  "confidence": 0.9
}
```

例如：

```
python app.py
python manage.py runserver
npm start
java -jar app.jar
python proxyPool.py server
python proxyPool.py schedule
```

Runtime Command 不一定用于 unit test，但用于 smoke test 或部署验证。

---

## 6.10 Verification 节点

Verification 节点表示最终验证方式。

例子：

```
{
  "id": "pytest --collect-only -q",
  "type": "verification",
  "evidence": ["tests/", "pytest.ini"],
  "confidence": 0.9
}
```

验证可以分层：

```
test discovery
build check
import smoke test
unit test
integration test
README command smoke test
```

建议先运行轻量验证，再运行完整验证。

例如：

```
pytest --collect-only -q
↓
pytest -q
```

或者：

```
cargo test --no-run
↓
cargo test
```

---

## 7. 边类型设计

Typed Task Graph 的边表示任务之间的依赖关系。

建议先支持以下几类边。

## 7.1 requires\_runtime

表示某个步骤依赖特定运行时。

```
{
  "from": "python>=3.10",
  "to": "poetry install",
  "type": "requires_runtime",
  "strength": "hard"
}
```

## 7.2 uses\_package\_manager

表示依赖安装应该通过哪个包管理器完成。

```
{
  "from": "poetry",
  "to": "poetry install",
  "type": "uses_package_manager",
  "strength": "hard"
}
```

## 7.3 system\_required\_by

表示某个系统库是某个语言包或构建步骤的底层依赖。

```
{
  "from": "libpq-dev",
  "to": "psycpg2",
  "type": "system_required_by",
  "strength": "soft"
}
```

## 7.4 dependency\_before\_build

表示依赖安装必须在项目构建之前完成。

```
{
  "from": "requirements.txt",
  "to": "pip install -e .",
  "type": "dependency_before_build",
  "strength": "hard"
}
```

## 7.5 build\_before\_verify

表示项目必须先安装或构建，才能执行测试。

```
{
  "from": "poetry install",
  "to": "pytest --collect-only -q",
  "type": "build_before_verify",
  "strength": "hard"
}
```

## 7.6 service\_required\_by

表示某个服务必须在测试或运行命令前启动。

```
{
  "from": "redis-server",
  "to": "pytest",
  "type": "service_required_by",
  "strength": "hard"
}
```

## 7.7 env\_required\_by

表示某个环境变量是运行或测试需要的。

```
{
  "from": "DATABASE_URL",
  "to": "pytest",
  "type": "env_required_by",
  "strength": "hard"
}
```

## 7.8 verify\_after\_runtime

表示 runtime command 启动后再进行 smoke test。

```
{
  "from": "python app.py",
  "to": "curl http://127.0.0.1:5000/health",
  "type": "verify_after_runtime",
  "strength": "hard"
}
```

## 8. 边的强度：hard edge 与 soft edge

Typed Task Graph 中的边建议区分为两类：

```
hard edge
soft edge
```

### 8.1 hard edge

`hard edge` 表示严格的前后依赖，必须参与拓扑排序。

例如：

```
{
  "from": "requirements-test.txt",
  "to": "pytest --collect-only -q",
  "type": "test_dependency_before_verify",
  "strength": "hard"
}
```

这表示必须先安装测试依赖，才能执行 pytest 验证。

### 8.2 soft edge

`soft edge` 表示经验性依赖或 fallback 依赖，不一定默认执行，但应该进入 risk notes 或 fallback plan。

例如：

```
{
  "from": "libxml2-dev",
  "to": "lxml",
  "type": "system_required_by",
  "strength": "soft"
}
```

这表示如果 `lxml` 安装失败，可能需要先安装 `libxml2-dev`，但不一定一开始就安装它。

### 8.3 默认处理方式

默认情况下：

```
hard edge 参与程序拓扑排序
soft edge 不直接决定主执行顺序
soft edge 进入 risk_notes 或 fallback_plan
```

如果某个 soft edge 被多次验证为必要依赖，可以升级为 hard edge。

---

## 9. Planning Agent 工作流程

Planning Agent 的执行流程分成 6 步。

---

### Step 1: 扫描仓库结构

先获得仓库整体结构。

重点寻找：

```
README
docs
Dockerfile
docker-compose.yml
.github/workflows
requirements.txt
requirements-test.txt
pyproject.toml
setup.py
setup.cfg
package.json
pom.xml
build.gradle
Cargo.toml
go.mod
Makefile
tox.ini
pytest.ini
tests/
```

输出：

```
仓库语言
候选构建工具
候选测试框架
候选配置文件
候选运行入口
```

---

### Step 2: 识别 Runtime 和 Package Manager

根据配置文件判断 repo 使用什么运行时和包管理器。

例如：



有 poetry.lock, 优先判断为 Poetry  
有 package-lock.json, 优先 npm  
有 pnpm-lock.yaml, 优先 pnpm  
有 pom.xml, 优先 Maven  
有 build.gradle, 优先 Gradle  
有 Cargo.toml, 优先 Cargo  
有 go.mod, 优先 Go modules

输出:

Runtime 节点  
Package Manager 节点

---

### Step 3: 抽取语言依赖和测试依赖

从 manifest 和 lockfile 中抽取依赖。

同时从测试目录中判断测试框架。

例如:

Python repo 中如果有 tests/、pytest.ini、conftest.py, 则推测 pytest  
JS repo 中如果 package.json 里有 scripts.test, 则记录 npm test  
Rust repo 中通常使用 cargo test  
Java repo 中根据 Maven / Gradle 判断测试命令

输出:

Language Dependency 节点  
Verification 节点

---

### Step 4: 识别系统依赖、服务、环境变量和数据资源

通过 README、CI、docker-compose 和代码搜索识别非语言包依赖。

重点搜索:

os.environ  
process.env  
getenv  
DATABASE\_URL  
REDIS\_URL

POSTGRES  
MYSQL  
MONGODB  
KAFKA  
RABBITMQ  
S3  
API\_KEY  
TOKEN  
CUDA  
ffmpeg  
opencv  
selenium  
playwright

典型规则：

如果发现 `import cv2`，可能需要 `libgl1`  
如果发现 `psycopg2`，可能需要 `libpq-dev`  
如果发现 `lxml`，可能需要 `libxml2-dev` / `libxslt1-dev`  
如果发现 `Redis` / `Postgres` / `MongoDB` 连接，可能需要 `service` 节点  
如果发现 `API key`，标记为 `env_var` 节点，不要让 `Build Agent` 盲目尝试

输出：

System Package 节点  
Service 节点  
Env Var / Secret 节点  
Data / Asset 节点  
Runtime Command 节点

## Step 5: 构建 Typed Task Graph

把前面识别出的所有节点组织成图。

一个简单顺序通常是：

Runtime / Base image  
↓  
System packages  
↓  
Package manager  
↓  
Language dependencies  
↓  
Project install/build  
↓

```
Runtime preparation
↓
Verification
```

但不是所有 repo 都需要每一层。

例如，一个简单 Python repo 的 graph 可能是：

```
python:3.11-slim
↓
pip
↓
requirements.txt
↓
requirements-test.txt
↓
pytest --collect-only -q
↓
pytest -q
```

一个需要 Redis 的 repo 可能还会有：

```
redis-server
↓
REDIS_URL
↓
pytest integration tests
```

## Step 6: 程序化生成 ordered todo-list

Planning Agent 不应该直接用 LLM 推理生成最终执行顺序。

更稳妥的做法是：

1. LLM 负责从 repo 中识别节点和边，生成 Typed Task Graph；
2. 程序负责校验 graph；
3. 程序对 graph 的 hard edges 做拓扑排序；
4. 程序根据节点类型优先级稳定排序；
5. soft edges 进入 risk notes 或 fallback plan；
6. 最终生成 ordered todo-list，交给 Build Agent 执行。

也就是说：

```
LLM 负责 repo 理解
程序负责排序正确性
```

整体流程是：

```
Repo files
↓
Planning Agent / LLM
↓
Typed Task Graph JSON
↓
Graph Validator
↓
Topological Sorter
↓
Ordered Todo-List
↓
Build Agent
```

## 10. Graph Validator 设计

在拓扑排序之前，程序需要先检查 graph 是否合法。

至少检查以下内容：

1. 每条 edge 的 `from` 和 `to` 必须都存在于 nodes 中；
2. 每个 node 必须有 `id` 和 `type`；
3. 每条 edge 必须有 `type` 和 `strength`；
4. 不允许出现重复 node id；
5. hard edges 组成的图不能有环；
6. soft edges 可以保留，但默认不参与主排序。

如果 graph 不合法，程序应该把错误反馈给 Planning Agent，让它修正 graph，而不是直接让 Build Agent 执行。

## 11. 节点类型优先级

在多个节点都可以执行时，程序应该根据节点类型设置默认优先级，避免排序结果随机变化。

建议优先级如下：

```
TYPE_PRIORITY = {
  "runtime": 0,
  "system_package": 1,
  "package_manager": 2,
  "language_dependency": 3,
  "project_build": 4,
  "service": 5,
  "env_var": 6,
```

```
"asset": 7,  
"runtime_command": 8,  
"verification": 9  
}
```

这个优先级表示：

```
先确定运行底座  
再准备系统依赖和包管理器  
然后安装语言依赖  
再构建项目  
最后启动运行时服务并验证
```

注意：

这个优先级只是 tie-breaker。真正决定先后关系的是 hard edges。

## 12. 拓扑排序算法

程序可以使用 Kahn 算法对 hard edges 做拓扑排序。

示例代码：

```
from collections import defaultdict  
  
TYPE_PRIORITY = {  
    "runtime": 0,  
    "system_package": 1,  
    "package_manager": 2,  
    "language_dependency": 3,  
    "project_build": 4,  
    "service": 5,  
    "env_var": 6,  
    "asset": 7,  
    "runtime_command": 8,  
    "verification": 9  
}  
  
def topo_sort_typed_graph(nodes, edges):  
    node_map = {node["id"]: node for node in nodes}  
  
    if len(node_map) != len(nodes):  
        raise ValueError("Duplicate node id found.")  
  
    graph = defaultdict(list)  
    indegree = {node["id"]: 0 for node in nodes}
```

```

for edge in edges:
    if edge.get("strength") != "hard":
        continue

    src = edge["from"]
    dst = edge["to"]

    if src not in node_map:
        raise ValueError(f"Edge source does not exist: {src}")
    if dst not in node_map:
        raise ValueError(f"Edge target does not exist: {dst}")

    graph[src].append(dst)
    indegree[dst] += 1

ready = [node_id for node_id, deg in indegree.items() if deg == 0]
result = []

while ready:
    ready.sort(
        key=lambda node_id: TYPE_PRIORITY.get(
            node_map[node_id].get("type"), 99
        )
    )

    current = ready.pop(0)
    result.append(current)

    for nxt in graph[current]:
        indegree[nxt] -= 1
        if indegree[nxt] == 0:
            ready.append(nxt)

if len(result) != len(nodes):
    raise ValueError("Hard-edge graph has a cycle.")

return result

```

这个函数输出的是排好序的 node id 列表。

## 13. 从排序结果生成 todo-list

拓扑排序得到的是 node id 顺序，程序还需要把每个 node 转换成 Build Agent 可执行的 todo item。

例如：

```

{
    "step": 1,

```

```
"node_id": "python:3.11-slim",
"task_type": "runtime",
"command_hint": "FROM python:3.11-slim",
"evidence": [".github/workflows/test.yml"],
"confidence": 0.9
}
```

Build Agent 执行时，不一定逐字照抄 `command_hint`，但必须尊重 todo-list 中的依赖顺序。

## 14. soft edges 的处理

soft edges 不参与默认主路径排序，而是进入 `risk_notes` 或 `fallback_plan`。

例如：

```
{
  "from": "libxml2-dev",
  "to": "lxml",
  "type": "system_required_by",
  "strength": "soft"
}
```

可以被转换成 fallback plan：

```
{
  "trigger": "pip install lxml fails with missing libxml2 or xslt headers",
  "suggested_action": "apt-get update && apt-get install -y libxml2-dev libxslt1-dev",
  "evidence": ["Dockerfile", "lxml dependency"],
  "confidence": 0.7
}
```

这样做的好处是：

Build Agent 不会一开始就安装所有可能有用的系统包，而是在失败后有针对性地 fallback。

## 15. 为什么不用 LLM 直接排序

不建议让 LLM 直接完成最终拓扑排序，因为 LLM 可能出现以下问题：

1. 漏掉某个前置依赖；
2. 把 verification 放到 dependency installation 之前；
3. 忽略 graph 中存在的 hard edge；
4. 生成看起来合理但不满足约束的执行顺序；
5. 在复杂 graph 中出现不稳定排序。

因此，最终排序应该由确定性程序完成。

LLM 可以参与：

节点识别  
边识别  
证据解释  
风险解释  
fallback 建议

但不应该承担：

排序正确性  
环检测  
边合法性检查  
最终执行顺序保证

## 16. 输出格式建议

Planning Agent 最终可以输出如下 JSON。

```
{
  "repo_summary": {
    "primary_language": "Python",
    "package_manager": "Poetry",
    "test_framework": "pytest",
    "recommended_base_image": "python:3.10-slim"
  },
  "typed_task_graph": {
    "nodes": [
      {
        "id": "python>=3.10",
        "type": "runtime",
        "evidence": ["pyproject.toml"],
        "confidence": 0.9
      },
      {
        "id": "poetry",
        "type": "package_manager",
        "evidence": ["poetry.lock"],
        "confidence": 0.95
      },
      {
        "id": "libpq-dev",
        "type": "system_package",
        "evidence": ["psycopg2 dependency"],
```



```

    "confidence": 0.7
  },
  {
    "id": "poetry install",
    "type": "project_build",
    "evidence": ["README.md", "poetry.lock"],
    "confidence": 0.9
  },
  {
    "id": "pytest --collect-only -q",
    "type": "verification",
    "evidence": ["tests/", "pytest.ini"],
    "confidence": 0.9
  }
],
"edges": [
  {
    "from": "python>=3.10",
    "to": "poetry",
    "type": "requires_runtime",
    "strength": "hard"
  },
  {
    "from": "poetry",
    "to": "poetry install",
    "type": "uses_package_manager",
    "strength": "hard"
  },
  {
    "from": "libpq-dev",
    "to": "poetry install",
    "type": "system_required_by",
    "strength": "soft"
  },
  {
    "from": "poetry install",
    "to": "pytest --collect-only -q",
    "type": "build_before_verify",
    "strength": "hard"
  }
]
},
"ordered_todo_list": [
  {
    "step": 1,
    "node_id": "python>=3.10",
    "task_type": "runtime",
    "command_hint": "FROM python:3.10-slim",
    "evidence": ["pyproject.toml"],
    "confidence": 0.9
  },

```

```

{
  "step": 2,
  "node_id": "poetry",
  "task_type": "package_manager",
  "command_hint": "pip install poetry",
  "evidence": ["poetry.lock"],
  "confidence": 0.95
},
{
  "step": 3,
  "node_id": "poetry install",
  "task_type": "project_build",
  "command_hint": "poetry install",
  "evidence": ["poetry.lock", "pyproject.toml"],
  "confidence": 0.9
},
{
  "step": 4,
  "node_id": "pytest --collect-only -q",
  "task_type": "verification",
  "command_hint": "poetry run pytest --collect-only -q",
  "evidence": ["tests/", "pytest.ini"],
  "confidence": 0.9
}
],
"risk_notes": [
  "Do not bypass poetry.lock with direct pip install unless poetry install fails and the failure is diagnosed.",
  "If psycpg2 build fails, check whether libpq-dev and build-essential are installed.",
  "If pytest is missing, check whether test dependencies are excluded by default."
],
"fallback_plan": [
  {
    "trigger": "psycpg2 build fails with missing pg_config",
    "suggested_action": "apt-get update && apt-get install -y libpq-dev build-essential",
    "evidence": ["psycpg2 dependency"],
    "confidence": 0.75
  }
],
"unresolved_questions": [
  "Whether integration tests require a real database service is unclear."
]
}

```

## 17. Build Agent 如何使用这个计划

Build Agent 不需要完全照搬 Planning Agent 的计划，但应该把它作为初始策略。

Build Agent 的执行原则是：

1. 优先执行高置信度步骤；
2. 遇到失败时，先检查该步骤的 risk\_notes；
3. 不要轻易绕过 lockfile；
4. 不要直接安装 random latest package；
5. 如果实际错误和 plan 不一致，则允许修改 task graph；
6. 每次修改 plan，都记录原因和证据；
7. 如果某个 soft edge 的 fallback 被验证为必要，可以把它升级为 hard edge。

也就是说，Planning Agent 输出的是初始地图，不是不可修改的死规则。

---

## 18. 简化版实现路线

第一阶段可以先实现 Python repo。

最小可行版本只需要支持：

1. 扫描文件结构；
2. 识别 Python 版本；
3. 识别 pip / poetry / conda；
4. 识别 requirements / pyproject / lockfile；
5. 识别 pytest；
6. 粗略识别 system packages；
7. 生成 typed task graph；
8. 程序校验 graph；
9. 程序对 hard edges 做拓扑排序；
10. 输出 ordered todo-list。

第二阶段再支持：

```
JavaScript / TypeScript  
Rust  
Java  
Go
```

第三阶段再加入：

```
服务识别  
环境变量识别  
数据资源识别  
runtime command 识别  
smoke test 生成  
fallback 自动升级
```

不要一开始就追求全语言、全资源、全场景。先把 typed task graph 的结构跑通。

---

## 19. 预期收益

Planning Agent 预期能带来三点收益。

第一，减少盲目试错。Build Agent 不再从 `pip install .` 或 `npm install` 开始乱试，而是根据 repo evidence 执行。

第二，减少环境污染。很多错误安装和错误 fallback 可以提前避免。

第三，提升可解释性。最终环境构建失败时，可以知道失败发生在 runtime、system package、language dependency、service、env var 还是 verification 阶段。

额外收益是：

因为 ordered todo-list 由程序拓扑排序生成，所以执行顺序更稳定，不依赖 LLM 的临场推理。

---

## 20. 核心设计原则

这个 Planning Agent 的核心不是“提前把所有依赖装好”，而是：

**在真正修改环境之前，先构造一个带类型、带证据、带依赖关系的环境任务图。**

Dependency graph 只是其中一部分。

完整环境搭建还必须考虑：

```
runtime
package manager
language dependencies
system packages
project build steps
services
environment variables
data assets
runtime commands
verification strategy
```

同时，最终执行顺序不应该由 LLM 直接生成，而应该由程序对 Typed Task Graph 的 hard edges 做拓扑排序得到。

因此，整个模块的关键分工是：

```
LLM: 理解 repo, 抽取 typed graph
程序: 校验 graph, 拓扑排序, 生成 todo-list
Build Agent: 根据 todo-list 执行并动态修正
```